

# Multivariate Analysis and Predictive Modeling of Digital Health Readiness in India

---

Interim Report

April 2026

Lakshya Gupta  
Sarthak Goel  
Krissh Modi

Dataset: Citizen Survey Dataset, India (Kalita et al., 2026)  
Harvard Dataverse doi:10.7910/DVN/M1DFB0  
50,217 households · 29 Indian states & UTs

## Contents

---

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction, Background and Motivation</b>                    | <b>3</b>  |
| 1.1      | Background . . . . .                                              | 3         |
| 1.2      | Motivation . . . . .                                              | 3         |
| 1.3      | Research Gap . . . . .                                            | 4         |
| <b>2</b> | <b>Problem Statement</b>                                          | <b>4</b>  |
| 2.1      | Research Objective . . . . .                                      | 4         |
| 2.2      | Variable Structure . . . . .                                      | 4         |
| <b>3</b> | <b>Technical Details: Algorithms, Methodology and Assumptions</b> | <b>5</b>  |
| 3.1      | Data Preprocessing . . . . .                                      | 5         |
| 3.2      | Feature Engineering: Digital Readiness Score (DRS) . . . . .      | 6         |
| 3.2.1    | Mathematical Formulation . . . . .                                | 6         |
| 3.2.2    | DRS Results . . . . .                                             | 6         |
| 3.3      | Full-Predictor PCA for Dimensionality Reduction . . . . .         | 6         |
| 3.4      | K-Means Clustering . . . . .                                      | 7         |
| 3.4.1    | Objective . . . . .                                               | 7         |
| 3.4.2    | Selection of K . . . . .                                          | 7         |
| 3.4.3    | Implementation Details . . . . .                                  | 7         |
| 3.5      | Logistic Regression . . . . .                                     | 8         |
| 3.6      | Random Forest . . . . .                                           | 8         |
| 3.7      | Gradient Boosting . . . . .                                       | 8         |
| 3.8      | Model Validation Strategy . . . . .                               | 9         |
| <b>4</b> | <b>Data Description and Source</b>                                | <b>9</b>  |
| 4.1      | Dataset . . . . .                                                 | 9         |
| 4.2      | Target Variable . . . . .                                         | 9         |
| 4.3      | Predictor Variable Mapping . . . . .                              | 9         |
| <b>5</b> | <b>Results of the Analysis</b>                                    | <b>10</b> |
| 5.1      | Stage 1: Digital Readiness Score (DRS) . . . . .                  | 10        |
| 5.2      | Stage 2: Full-Predictor PCA and K-Means Clustering . . . . .      | 10        |
| 5.2.1    | Cluster Profiles . . . . .                                        | 10        |
| 5.3      | Stage 3: Logistic Regression . . . . .                            | 10        |
| 5.4      | Stage 3: Random Forest Feature Importance . . . . .               | 11        |
| 5.5      | Comparative Model Performance . . . . .                           | 11        |
|          | <b>Figures</b>                                                    | <b>12</b> |
| <b>6</b> | <b>Conclusions and Key Observations</b>                           | <b>16</b> |
| <b>7</b> | <b>Overall Learning Outcomes</b>                                  | <b>17</b> |
| <b>8</b> | <b>Limitations of the Proposed Study</b>                          | <b>17</b> |
| <b>9</b> | <b>Future Directions and Scope for Further Investigation</b>      | <b>18</b> |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>A Complete Python Code</b>                                       | <b>20</b> |
| <b>B History of AI Prompts and Chats</b>                            | <b>24</b> |
| <b>C AI-Code Correspondence: One-to-One Mapping</b>                 | <b>26</b> |
| <b>D Additional Technical Details and Supplementary Information</b> | <b>28</b> |

**Abstract**

This interim report documents the implementation and results of a three-stage multivariate analytical pipeline designed to model digital health readiness among Indian citizens. Using the *Citizen Survey Dataset, India* (Kalita et al., 2026)—a household-level survey covering 50,217 respondents across 29 states and Union Territories—we apply Principal Component Analysis (PCA) for dimensionality reduction, K-Means clustering for unsupervised population segmentation, and three comparative classifiers (Logistic Regression, Random Forest, Gradient Boosting) for predictive modelling of telemedicine usage. The analytic sample comprised 30,629 respondents with valid target observations. Three population segments were identified. Gradient Boosting achieved the highest ROC-AUC of **0.9309**; however, the AUC gap over Logistic Regression was only **0.0195**, supporting preference for the parametric model on interpretability grounds. Digital behaviour variables and the engineered Digital Readiness Score (DRS) emerged as the dominant predictors of telemedicine usage.

## 1. Introduction, Background and Motivation

### 1.1. Background

Digital health technologies—encompassing telemedicine platforms, mobile health applications, and online medical consultation services—are rapidly reshaping healthcare delivery systems worldwide. In India, a country of continental scale with profound geographical, economic, and infrastructural heterogeneity, these technologies carry particular transformative potential. Rural populations frequently face critical shortfalls in specialist availability, and digital platforms represent a viable mechanism for extending care beyond physical infrastructure constraints.

The COVID-19 pandemic catalysed an unprecedented acceleration in telehealth adoption globally [2]. Regulatory reforms in India—most notably the Telemedicine Practice Guidelines of 2020—legitimised the remote delivery of medical consultations and enabled broader institutional uptake of virtual care [4]. Yet despite these structural tailwinds, adoption remains deeply unequal.

### 1.2. Motivation

The adoption of digital health technologies is not determined by any single factor in isolation. Rather, it emerges from the *simultaneous configuration* of socio-demographic and infrastructural variables: age, income, education, gender, residential type, internet access, device ownership, and digital literacy act jointly. A low-income rural individual with smartphone access faces fundamentally different barriers to telemedicine than an urban high-income individual with equivalent connectivity.

This multivariate nature of the problem is the central motivation for our analytical framework. Prior work—largely conducted in Western healthcare contexts [3]—has predominantly relied on univariate and bivariate analyses, which are inadequate for capturing interaction effects, latent population structure, and the joint predictive influence of correlated variables.

**Core insight:** No single variable explains telemedicine readiness. It is the joint, simultaneous configuration of demographic and digital-infrastructure factors that determines whether an individual is positioned to benefit from digital health services.

### 1.3. Research Gap

While existing literature identifies socio-economic inequalities in digital health adoption, there is limited work—particularly in the Indian setting—that applies a unified multivariate pipeline combining dimensionality reduction, unsupervised clustering, and comparative supervised classification. This study bridges this gap by constructing a coherent three-stage pipeline in which each method’s output feeds the next.

## 2. Problem Statement

---

### 2.1. Research Objective

This project models and analyses telemedicine readiness among Indian citizens using a unified multivariate statistical pipeline comprising three connected stages:

- 1. Dimensionality Reduction (PCA):** Reduce the correlated multivariate predictor space into interpretable latent components capturing the dominant axes of digital readiness variation.
- 2. Unsupervised Clustering (K-Means):** Apply clustering on PCA-derived components to identify distinct, data-driven population segments defined by combined socio-demographic profiles.
- 3. Supervised Classification:** Model telemedicine usage as a binary outcome using Logistic Regression, Random Forest, and Gradient Boosting. Compare parametric and non-parametric approaches on predictive performance and interpretability.

### 2.2. Variable Structure

The full variable set used across all analytical stages is defined in Table 1.

**Table 1.** Variable structure and dataset mapping

| Role         | Variable                  | Dataset Code | Type / Description                                   |
|--------------|---------------------------|--------------|------------------------------------------------------|
| Response (Y) | Telemedicine usage        | c17_f        | Binary (1 = Great Extent/-Somewhat, 0 = Very Little) |
| Predictor    | Age                       | a2b_age      | Continuous (years)                                   |
| Predictor    | Income proxy              | a2g_food     | Continuous (food expenditure, INR)                   |
| Predictor    | Education level           | rec_a2d      | Ordinal (1=Primary, ..., 4=Higher-Sec+)              |
| Predictor    | Gender                    | a2c          | Binary (Female=1, Male=0)                            |
| Predictor    | Residential type          | residence    | Binary (Rural=1, Urban=0)                            |
| Predictor    | Internet as health source | c16_g        | Binary                                               |
| Predictor    | Health info usage         | c17_a        | Binary (recoded)                                     |
| Predictor    | E-pharmacy usage          | c17_b        | Binary (recoded)                                     |
| Predictor    | Find doctors online       | c17_c        | Binary (recoded)                                     |
| Predictor    | Access medical records    | c17_d        | Binary (recoded)                                     |
| Predictor    | Treatment help online     | c17_e        | Binary (recoded)                                     |
| Predictor    | Health card importance    | c18_a        | Binary                                               |
| Eng. Feature | Digital Readiness Score   | DRS          | Continuous (PCA composite)                           |
| Interaction  | Gender $\times$ Rural     | —            | Binary product                                       |
| Interaction  | Income $\times$ Internet  | —            | Continuous $\times$ Binary                           |

### 3. Technical Details: Algorithms, Methodology and Assumptions

#### 3.1. Data Preprocessing

The raw dataset contained 50,217 respondents and 232 variables. Preprocessing involved:

1. **Target Encoding:** The target variable c17\_f (frequency of video-call consultations) was recoded as binary: *Great Extent* and *Somewhat*  $\rightarrow$  1; *Very Little*  $\rightarrow$  0. Responses of *DK* (Don't Know) and *DNA* (Does Not Apply) were treated as missing and excluded from the analytic sample. This yielded 30,629 valid observations.
2. **Missing Value Imputation:** Continuous predictors (a2b\_age, a2g\_food) were imputed with the column median; ordinal and binary predictors were imputed with the column mode (`SimpleImputer`, `scikit-learn`).
3. **Standardisation:** All continuous predictors were *z*-score standardised (zero mean, unit variance) prior to PCA and K-Means, both of which are sensitive to feature scale.

**Assumption:** Missingness in predictors is treated as Missing Completely at Random (MCAR) for imputation purposes. Systematic missingness patterns were noted but not modelled probabilistically in this iteration.

### 3.2. Feature Engineering: Digital Readiness Score (DRS)

A composite **Digital Readiness Score (DRS)** was engineered from six digital-behaviour indicators: c16\_g, c17\_a, c17\_b, c17\_c, c17\_d, c17\_e.

#### 3.2.1. Mathematical Formulation

Let  $\mathbf{X}_{\text{drs}} \in \mathbb{R}^{n \times 6}$  be the standardised matrix of these six indicators. The covariance matrix is decomposed as:

$$\Sigma = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top \quad (1)$$

where  $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_6)$  contains eigenvalues in descending order, and the columns of  $\mathbf{V}$  are the corresponding eigenvectors. The primary DRS construction:

$$\text{DRS}_i = \mathbf{v}_1^\top \mathbf{x}_i \quad (2)$$

where  $\mathbf{x}_i$  is the standardised feature vector for individual  $i$ . **Fallback:** If PC1 explains less than 60% of variance, a variance-weighted composite is used instead:

$$\text{DRS}_i = \sum_{k=1}^6 \frac{\lambda_k}{\sum_j \lambda_j} \mathbf{v}_k^\top \mathbf{x}_i \quad (3)$$

#### 3.2.2. DRS Results

PC1 explained **54.5%** of variance (below the 60% threshold), so the variance-weighted composite was used. The dominant PC1 loadings were:

**Table 2.** PC1 loadings for Digital Readiness Score construction

| Variable | Description                              | PC1 Loading |
|----------|------------------------------------------|-------------|
| c17_c    | Uses internet to find doctors/facilities | +0.4643     |
| c17_d    | Uses internet to access medical records  | +0.4598     |
| c17_b    | Uses internet for e-pharmacies           | +0.4399     |
| c17_a    | Uses internet for general health info    | +0.4388     |
| c16_g    | Internet as health information source    | +0.3269     |
| c17_e    | Uses internet for treatment help         | +0.2829     |

### 3.3. Full-Predictor PCA for Dimensionality Reduction

A second PCA was applied to the full set of 12 predictors to reduce dimensionality before clustering. Let  $\mathbf{X} \in \mathbb{R}^{n \times p}$  be the standardised design matrix. The sample covariance matrix is:

$$\Sigma = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X} \quad (4)$$

The retained projection matrix  $\mathbf{V}_d \in \mathbb{R}^{p \times d}$  comprises the top  $d$  eigenvectors explaining at least 80% of cumulative variance. The projected data:

$$\mathbf{Z} = \mathbf{X}\mathbf{V}_d, \quad \mathbf{Z} \in \mathbb{R}^{n \times d} \quad (5)$$

The algorithm retained  $d = 8$  components to exceed the 80% cumulative variance threshold.

### 3.4. K-Means Clustering

K-Means was applied to  $\mathbf{Z}$  to identify latent user segments without prior assumptions about group membership.

#### 3.4.1. Objective

Partition  $n$  observations into  $K$  clusters  $\{C_1, \dots, C_K\}$  minimising the Within-Cluster Sum of Squares (WCSS):

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{z}_i - \boldsymbol{\mu}_k\|^2 \quad (6)$$

where  $\mathbf{z}_i \in \mathbb{R}^d$  is the PCA projection of individual  $i$  and  $\boldsymbol{\mu}_k$  is the centroid of cluster  $k$ .

#### 3.4.2. Selection of $K$

The optimal number of clusters  $K$  was selected jointly via:

- **Elbow Method:** Plot of WCSS against  $K$ ; identifies inflection point.
- **Silhouette Analysis:** For observation  $i$ , the silhouette coefficient is:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (7)$$

where  $a(i)$  is the mean intra-cluster distance and  $b(i)$  is the mean distance to the nearest neighbouring cluster.  $s(i) \in [-1, 1]$ ; values near +1 indicate well-separated clusters.

The optimal  $K$  was **3**, corresponding to the highest mean silhouette score of **0.2975**.

#### 3.4.3. Implementation Details

K-Means used the `k-means++` initialisation strategy with  $n\_init = 20$  random restarts and a maximum of 500 iterations per restart, ensuring convergence to a stable solution. A random subsample (up to 5,000 observations) was used for silhouette computation to manage computational cost.



### 3.5. Logistic Regression

Telemedicine usage  $Y \in \{0, 1\}$  was modelled as:

$$P(Y = 1 \mid \mathbf{X}) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top \mathbf{X})}} \quad (8)$$

The coefficient vector  $\beta \in \mathbb{R}^p$  is estimated by maximum likelihood:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log P(Y = 1 \mid \mathbf{x}_i) + (1 - y_i) \log(1 - P(Y = 1 \mid \mathbf{x}_i))] \quad (9)$$

The full feature vector included all 12 predictors, the DRS, two interaction terms (Gender  $\times$  Rural; Income  $\times$  Internet), and two cluster dummy variables—totalling  $p = 17$  features. Coefficients are exponentiated to yield *odds ratios*  $e^{\hat{\beta}_j}$ : the multiplicative change in odds of telemedicine usage per unit increase in predictor  $j$ , holding all others constant. The model used  $C = 1.0$  regularisation strength (L2), the `lbfgs` solver, and `class_weight='balanced'` to address the 35/65 class imbalance.

### 3.6. Random Forest

The Random Forest ensembles  $B = 200$  decision trees built on bootstrap samples of the training data:

$$\hat{Y} = \text{mode}(T_1(\mathbf{X}), T_2(\mathbf{X}), \dots, T_B(\mathbf{X})) \quad (10)$$

Each tree  $T_b$  is trained on a bootstrap sample, and each node split considers only  $m = \lfloor \sqrt{p} \rfloor$  randomly selected features, introducing variance reduction via both bagging and feature randomisation.

**Hyperparameters:** `n_estimators=200`, `max_depth=8`, `max_features='sqrt'`, `class_weight='balanced'`, `oob_score=True`.

### 3.7. Gradient Boosting

Gradient Boosting builds trees sequentially, each correcting the pseudo-residuals of the prior ensemble:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x}) \quad (11)$$

where  $\eta$  is the learning rate,  $h_m$  is the  $m$ -th tree fitted to the negative gradient of the loss, and  $F_0$  is a constant initialisation (log-odds of the base rate). Stochastic gradient boosting (`subsample=0.8`) introduces additional variance reduction.

**Hyperparameters:** `n_estimators=200`, `learning_rate=0.05`, `max_depth=4`, `subsample=0.8`.

### 3.8. Model Validation Strategy

1. **Train-test split:** 80/20 stratified split preserving class balance.
2. **5-Fold Cross-Validation:** Stratified K-Fold on the training set for performance estimation.
3. **Bootstrap Resampling:**  $B = 1000$  bootstrap resamples of the test set to compute 95% confidence intervals on ROC-AUC.

The primary performance metric is **ROC-AUC** due to the 35/65 class imbalance in the analytic sample.

## 4. Data Description and Source

### 4.1. Dataset

#### Dataset: Citizen Survey Dataset, India (Kalita et al., 2026)

|                        |                                      |
|------------------------|--------------------------------------|
| <b>Source</b>          | Harvard Dataverse                    |
| <b>DOI</b>             | doi:10.7910/DVN/M1DFB0               |
| <b>Coverage</b>        | 29 Indian States & Union Territories |
| <b>Raw size</b>        | 50,217 households, 232 variables     |
| <b>Survey year</b>     | 2022–2023                            |
| <b>Analytic sample</b> | 30,629 (valid target observations)   |

### 4.2. Target Variable

The response variable is derived from survey question c17\_f: “*To what extent do you use video-call or online consultation services for healthcare?*” Responses were recoded as:

- $Y = 1$  (Telemedicine user): *Great Extent* or *Somewhat*
- $Y = 0$  (Non-user): *Very Little*
- *DK / DNA*: Excluded (missing target)

The analytic sample showed a **34.6%** telemedicine usage prevalence (10,585 users; 20,044 non-users).

### 4.3. Predictor Variable Mapping

Due to the absence of exact proposed variables in the Stata dataset, the following operationalisation decisions were made:

**Table 3.** Mapping from proposal variables to dataset columns

| Proposed Variable        | Dataset Column  | Rationale                             |
|--------------------------|-----------------|---------------------------------------|
| Monthly household income | a2g_food        | Food expenditure as income proxy      |
| Internet access          | c16_g           | Internet as health information source |
| Smartphone ownership     | c17_a–c17_e     | Digital health behaviour proxies      |
| Digital literacy         | Composite (DRS) | PCA-derived from 6 digital indicators |
| Health app usage         | c18_a           | Health card / digital records usage   |

## 5. Results of the Analysis

### 5.1. Stage 1: Digital Readiness Score (DRS)

PCA over the six digital-behaviour indicators produced variance shares:

**PC1–PC6 variance explained:** [54.5%, 13.5%, 12.1%, 7.6%, 6.9%, 5.3%]

Since PC1 (54.5%) fell below the 60% threshold specified in the proposal, the variance-weighted composite DRS was constructed. The four strongest PC1 loadings (c17\_c: +0.464; c17\_d: +0.460; c17\_b: +0.440; c17\_a: +0.439) indicate that **active digital health behaviours**—finding providers, accessing records, e-pharmacy, health information—dominate the latent readiness structure.

### 5.2. Stage 2: Full-Predictor PCA and K-Means Clustering

The full-predictor PCA retained  $d = 8$  **components** to exceed 80% cumulative variance. K-Means on the  $\mathbf{Z} \in \mathbb{R}^{n \times 8}$  projection identified  $K = 3$  **clusters** as optimal (max silhouette = 0.2975).

#### 5.2.1. Cluster Profiles

**Table 4.** K-Means cluster composition and telemedicine usage rates

| Cluster | Interpretation                        | N      | % of Sample | Telemedicine Rate |
|---------|---------------------------------------|--------|-------------|-------------------|
| 0       | Intermediate readiness                | 2,205  | 7.2%        | 37.7%             |
| 1       | High readiness (digitally engaged)    | 19,694 | 64.3%       | 46.9%             |
| 2       | Low readiness (digitally constrained) | 8,730  | 28.5%       | 5.9%              |

**Key finding:** Cluster 2 (28.5% of the sample, *digitally constrained*) had a telemedicine usage rate of only **5.9%**, compared to **46.9%** in Cluster 1 (*digitally engaged*)—an 8× difference in adoption probability driven by the joint profile of digital access and behaviour variables.

### 5.3. Stage 3: Logistic Regression

The logistic regression model achieved an ROC-AUC of **0.9114** on the holdout set. Table 5 reports the odds ratios, sorted by effect magnitude.

**Table 5.** Logistic regression odds ratios (sorted by distance from OR=1)

| Feature                         | Direction  | Odds Ratio |
|---------------------------------|------------|------------|
| c17_e (treatment help online)   | ↑ Positive | 4.2934     |
| DRS (Digital Readiness Score)   | ↑ Positive | 1.5654     |
| cluster_2 (constrained segment) | ↓ Negative | 0.7082     |
| income_x_internet               | ↑ Positive | 1.2269     |
| c17_d (medical records online)  | ↑ Positive | 1.1969     |
| education                       | ↑ Positive | 1.1638     |
| rural                           | ↑ Positive | 1.1345     |
| c17_b (e-pharmacy)              | ↑ Positive | 1.1282     |
| c18_a (health card)             | ↓ Negative | 0.8781     |
| gender (female)                 | ↑ Positive | 1.1030     |
| gender_x_rural                  | ↓ Negative | 0.9059     |
| c16_g (internet source)         | ↑ Positive | 1.0546     |
| a2b_age                         | ↑ Positive | 1.0048     |

#### 5.4. Stage 3: Random Forest Feature Importance

**Table 6.** Top-10 Random Forest feature importances (mean decrease in impurity)

| Feature                       | Importance |
|-------------------------------|------------|
| c17_e (treatment help online) | 0.4095     |
| DRS                           | 0.2825     |
| cluster_2                     | 0.0775     |
| education                     | 0.0711     |
| c17_d (medical records)       | 0.0333     |
| cluster_1                     | 0.0258     |
| c17_c (find doctors)          | 0.0234     |
| income_x_internet             | 0.0141     |
| a2b_age                       | 0.0126     |
| a2g_food                      | 0.0119     |

#### 5.5. Comparative Model Performance

**Table 7.** Holdout set performance (20% stratified test set,  $n = 6,126$ )

| Model               | Accuracy      | Precision     | Recall | F1     | ROC-AUC       |
|---------------------|---------------|---------------|--------|--------|---------------|
| Logistic Regression | 0.8779        | 0.8334        | 0.8082 | 0.8206 | 0.9114        |
| Random Forest       | 0.8800        | 0.8348        | 0.8139 | 0.8242 | 0.9306        |
| Gradient Boosting   | <b>0.8810</b> | <b>0.8491</b> | 0.7974 | 0.8224 | <b>0.9309</b> |

**Table 8.** 5-Fold CV ROC-AUC on training set ( $n = 24,503$ )

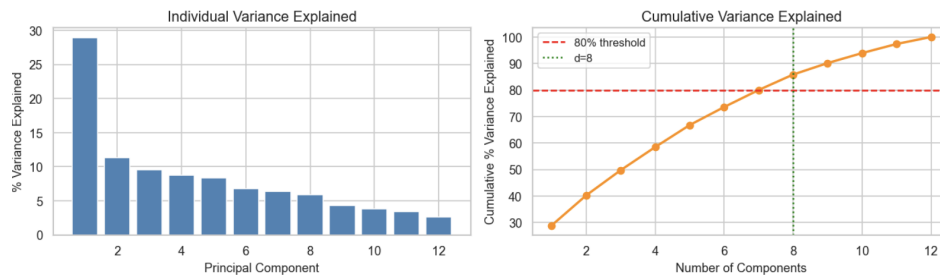
| Model               | Mean AUC      | Std. Dev.    |
|---------------------|---------------|--------------|
| Logistic Regression | 0.9052        | $\pm 0.0035$ |
| Random Forest       | 0.9245        | $\pm 0.0018$ |
| Gradient Boosting   | <b>0.9253</b> | $\pm 0.0014$ |

**Table 9.** Bootstrap 95% confidence intervals for ROC-AUC ( $B = 1000$  resamples)

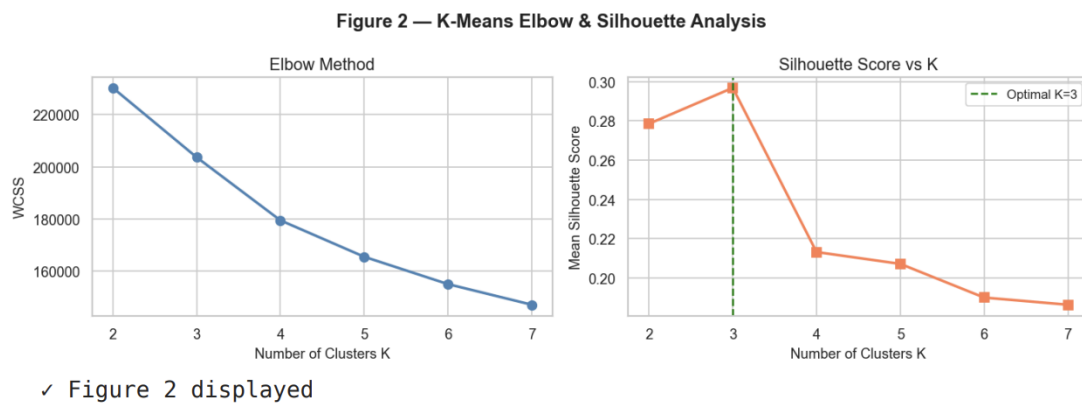
| Model               | AUC           | 95% CI (lower) | 95% CI (upper) |
|---------------------|---------------|----------------|----------------|
| Logistic Regression | 0.9114        | 0.9033         | 0.9192         |
| Random Forest       | 0.9306        | 0.9236         | 0.9369         |
| Gradient Boosting   | <b>0.9309</b> | 0.9240         | 0.9372         |

## Figures

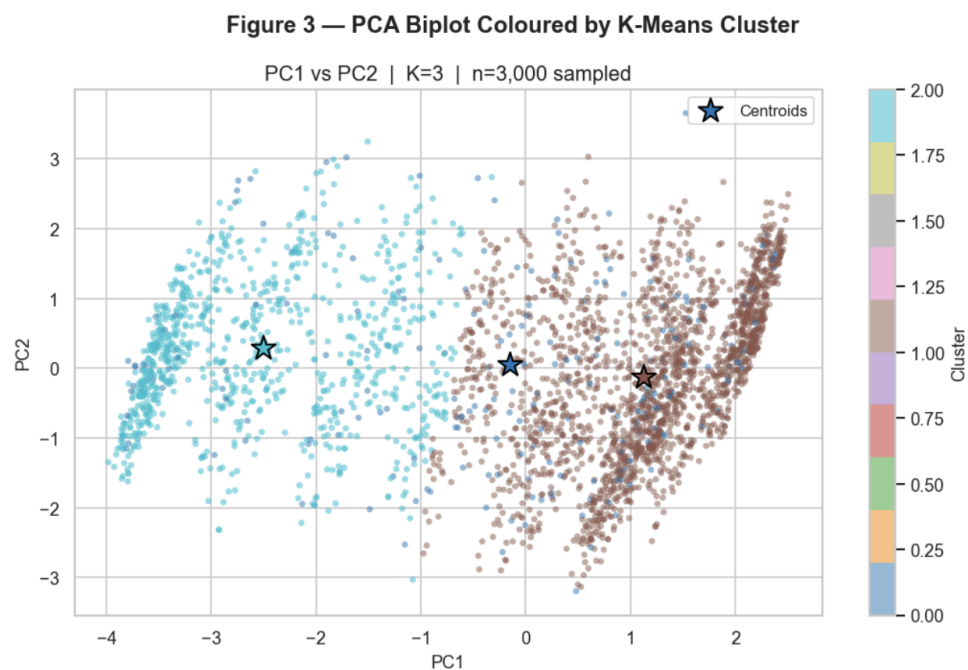
Export figures from the notebook using `plt.savefig("figN.pdf", bbox_inches="tight", dpi=150)` before each `plt.show()` call, and place the PDFs in the same folder as this `.tex` file.

**Figure 1.** PCA scree plot for DRS construction over 6 digital-behaviour indicators.

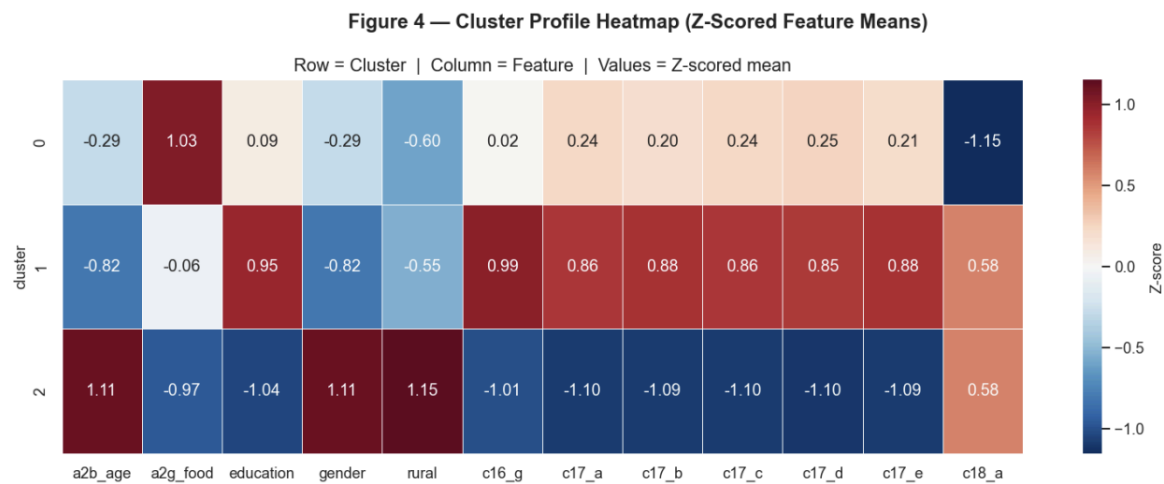
PC1 explains 54.5% of variance; since this is below the 60% threshold, the variance-weighted composite is used as the DRS.



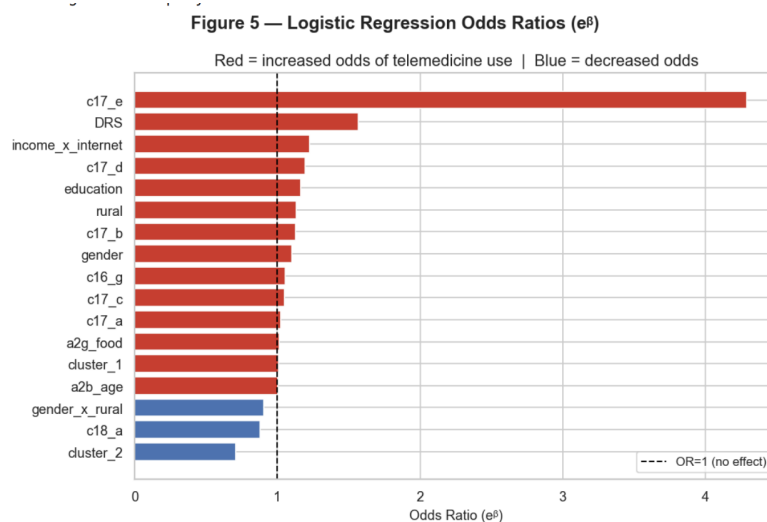
**Figure 2.** K-Means cluster selection. *Left:* Elbow method — WCSS flattens after  $K = 3$ . *Right:* Silhouette score peaks at  $K = 3$  (score = 0.2975), confirming the optimal number of clusters.



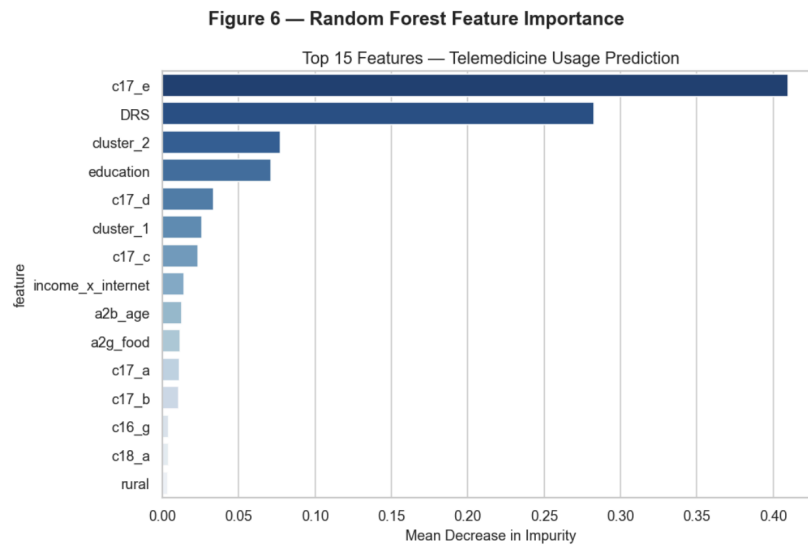
**Figure 3.** PCA biplot (PC1 vs PC2) coloured by K-Means cluster assignment ( $n = 3,000$  sampled). Cluster 2 (digitally constrained) separates clearly along the PC1 axis.



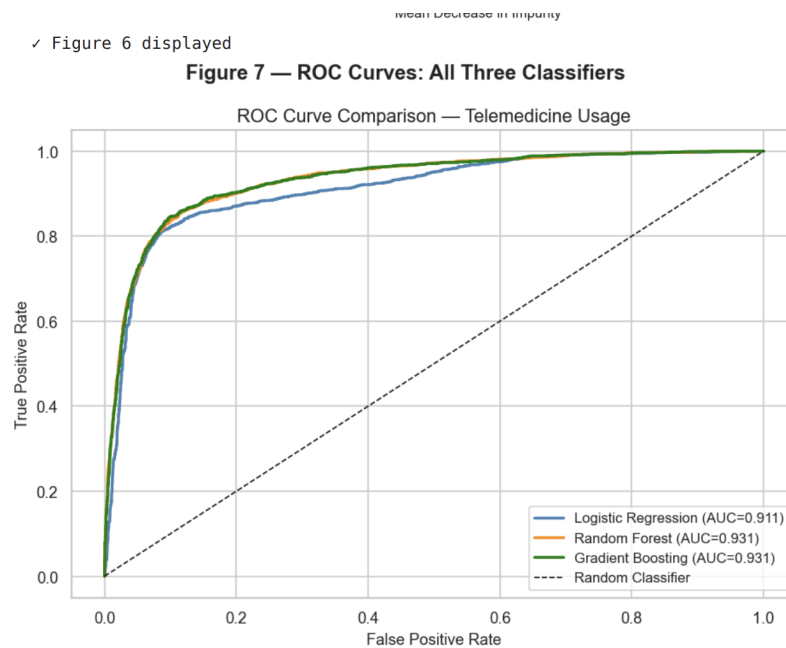
**Figure 4.** Cluster profile heatmap (Z-scored feature means, rows = clusters, columns = features). Cluster 2 shows strongly negative Z-scores across all digital behaviour variables.



**Figure 5.** Logistic regression odds ratios ( $e^{\beta}$ ). Red bars: positive association with telemedicine usage; blue bars: negative association. c17\_e (OR = 4.29) and DRS (OR = 1.57) are the dominant predictors.

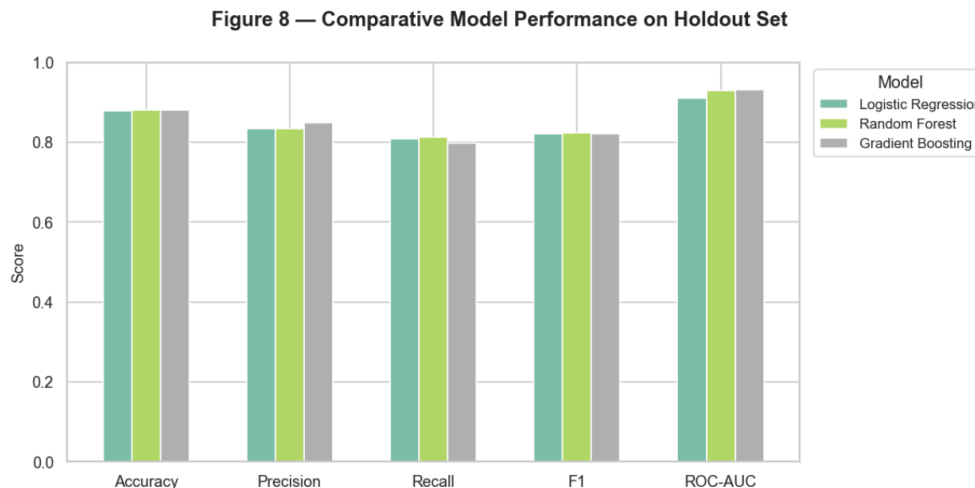


**Figure 6.** Random Forest feature importances (mean decrease in impurity, top 15 features). c17\_e (0.41) and DRS (0.28) together account for nearly 70% of total importance.



**Figure 7.** ROC curves for all three classifiers on the holdout test set ( $n = 6,126$ ). Gradient Boosting (AUC = 0.9309) and Random Forest (AUC = 0.9306) are nearly identical; Logistic Regression (AUC = 0.9114) remains highly competitive.





**Figure 8.** Comparative model performance across all five metrics on the holdout test set. All three models perform closely; Gradient Boosting leads marginally on ROC-AUC.

## 6. Conclusions and Key Observations

- C1. Digital behaviour dominates structural predictors.** The variable `c17_e` (using the internet for treatment help) showed the largest odds ratio of **4.29** in logistic regression and accounted for 41% of total feature importance in Random Forest. Digital behaviour variables consistently outperform socio-demographic predictors across all models.
- C2. Three distinct population segments.** K-Means identified three clusters: a digitally engaged majority (64.3%) with 46.9% telemedicine uptake, a constrained minority (28.5%) with only 5.9% uptake, and a small intermediate group (7.2%) with 37.7% uptake. This reveals stark within-population inequality in digital health access.
- C3. DRS is an effective summary feature.** The engineered Digital Readiness Score (OR = 1.57; RF importance = 0.28) is the second-strongest predictor, validating the PCA-based feature engineering approach.
- C4. Non-linear models marginally outperform, but LR is preferred.** Gradient Boosting achieved the highest AUC (0.9309), but the AUC gap over Logistic Regression was **0.0195**—below the 0.03 threshold for strong non-linear advantage. The parametric model is therefore recommended for its policy interpretability.
- C5. Interaction effects confirmed.** The `income_x_internet` interaction (OR = 1.23) confirms that internet access amplifies income-driven readiness. The `gender_x_rural` interaction (OR = 0.91) indicates a gender penalty concentrated in rural settings.
- C6. Unexpected positive association with rural residence.** Rural respondents show a positive coefficient for telemedicine usage (OR = 1.13), possibly reflecting the importance of telemedicine as a substitute for absent in-person care in rural areas.

## 7. Overall Learning Outcomes

- **PCA:** Applied eigendecomposition of the sample covariance matrix to both a focused feature set (DRS construction) and the full predictor space (dimensionality reduction before clustering). Understood the scree plot, cumulative variance rule, and the decision to use fallback aggregation when PC1 variance falls below a threshold.
- **K-Means Clustering:** Implemented the full K-Means pipeline including `k-means++` initialisation, elbow method, and silhouette analysis for  $K$  selection. Interpreted cluster centroids as population segment profiles with real-world socio-demographic meaning.
- **Logistic Regression:** Derived and implemented maximum likelihood estimation for binary classification. Computed and interpreted odds ratios as a policy-readable effect size. Applied interaction terms to capture conditional effects.
- **Random Forest:** Understood bootstrap aggregation (bagging) and feature randomisation. Computed out-of-bag (OOB) accuracy and mean decrease in impurity (MDI) feature importances.
- **Gradient Boosting:** Understood sequential residual-fitting and the role of learning rate  $\eta$ , tree depth, and subsampling in controlling model complexity.
- **Model Validation:** Applied 5-fold cross-validation and  $B = 1000$  bootstrap resampling for robust AUC estimation with 95% confidence intervals. Understood the importance of stratification in the presence of class imbalance.
- **Pipeline Integration:** Implemented a coherent analytical pipeline where PCA output feeds K-Means, cluster labels feed the classifiers, and all stages share a common variable set—demonstrating the value of a unified multivariate framework over sequential univariate analyses.

## 8. Limitations of the Proposed Study

- L1. Proxy variable substitution.** Several key variables in the proposal (monthly household income in INR, smartphone ownership, digital literacy score) were not available in the dataset under their intended form. Proxy substitutions—food expenditure for income, digital behaviour indicators for literacy—introduce measurement error and may not fully capture the intended constructs.
- L2. Target variable operationalisation.** The target `c17_f` measures *frequency* of video-call consultations rather than a direct telemedicine usage indicator. The recoding threshold (“Very Little” = 0; “Somewhat/Great Extent” = 1) is a discretisation judgment that may affect the class boundary and misclassify marginal users.
- L3. Exclusion of ambiguous responses.** Approximately 38.9% of the raw sample (19,588 observations) were excluded due to “DK” or “DNA” responses on the target variable. If non-response is systematically correlated with digital exclusion, this could underrepresent the most constrained populations.
- L4. Cross-sectional design.** The survey is cross-sectional; causal inference is not possible. Observed associations between digital behaviour and telemedicine usage may

reflect reverse causality (prior telemedicine experience prompting digital adoption).

- L5. Silhouette values.** The maximum silhouette score of 0.2975 is relatively modest, indicating that while  $K = 3$  is optimal among tested values, the clusters are not perfectly separated in PCA space. Cluster boundaries should be interpreted as soft rather than hard.
- L6. MCAR assumption.** Missing values in predictors were imputed assuming MCAR. Systematic missingness (e.g., rural respondents less likely to answer digital behaviour questions) was not formally modelled.

## 9. Future Directions and Scope for Further Investigation

---

- F1. Multiple imputation.** Replace single-value imputation with multiple imputation by chained equations (MICE) to propagate uncertainty from missing values into the inferential pipeline.
- F2. Probabilistic cluster models.** Apply Gaussian Mixture Models (GMMs) in place of K-Means to obtain soft cluster assignments with uncertainty quantification, better suited to the overlapping cluster structure evidenced by silhouette scores.
- F3. SHAP explanations.** Implement SHAP (SHapley Additive exPlanations) values for the Random Forest and Gradient Boosting models to recover local, feature-level explanations comparable to logistic regression odds ratios.
- F4. Spatial analysis.** Integrate state-level geographic variation to examine whether digital health inequalities exhibit spatial clustering (Moran's  $I$ ), and to identify states where targeted policy interventions would have greatest impact.
- F5. Longitudinal extension.** If repeated-wave survey data become available, apply panel models or difference-in-differences designs to establish temporal trends in digital readiness and assess the impact of policy changes.
- F6. Policy simulation.** Use the fitted logistic regression as a structural model to simulate counterfactual scenarios: what would telemedicine adoption rates look like if rural internet penetration increased by 20%, or if digital literacy programmes reached the constrained cluster?
- F7. Intersectionality analysis.** Extend the model to examine three-way interactions (e.g., Gender  $\times$  Rural  $\times$  Income) to quantify intersectional penalties facing low-income rural women.

## References

---

- [1] Kalita, A., et al. (2026). *Citizen Survey Dataset, India*. Harvard Dataverse. <https://doi.org/10.7910/DVN/M1DFB0>
- [2] Keesara, S., Jonas, A., & Schulman, K. (2020). Covid-19 and health care's digital revolution. *New England Journal of Medicine*, 382(23), e82. <https://doi.org/10.1056/NEJMp2005835>

- [3] Nouri, S., Khoong, E. C., Lyles, C. R., & Karliner, L. (2020). Addressing equity in telemedicine for chronic disease management during the Covid-19 pandemic. *NEJM Catalyst Innovations in Care Delivery*, 1(3). <https://doi.org/10.1056/CAT.20.0123>
- [4] Wosik, J., Fudim, M., Cameron, B., Gellad, Z. F., Cho, A., Phinney, D., ... & Tchong, J. (2020). Telehealth transformation: COVID-19 and the rise of virtual care. *Journal of the American Medical Informatics Association*, 27(6), 957–962. <https://doi.org/10.1093/jamia/ocaa067>

## A. Complete Python Code

The following pages contain the complete Python implementation of the pipeline. The code is executed in a single Jupyter Notebook (CADS\_Project.ipynb).

**Saving figures:** To embed figures in this report, add the following line immediately before each `plt.show()` call in Step 9 of the notebook: `plt.savefig("figN_name.pdf", bbox_inches="tight", dpi=150)` using the file-names: `fig1_drs_scee.pdf`, `fig2_kmeans_selection.pdf`, `fig3_pca_biplot.pdf`, `fig4_cluster_heatmap.pdf`, `fig5_odds_ratios.pdf`, `fig6_rf_importance.pdf`, `fig7_roc_curves.pdf`, `fig8_model_comparison.pdf`. Place all PDFs in the same directory as `interim_report_CADS.tex`.

### A.1 — Imports and Configuration

```

1 import warnings
2 warnings.filterwarnings("ignore")
3
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 from sklearn.preprocessing import StandardScaler, OrdinalEncoder
10 from sklearn.decomposition import PCA
11 from sklearn.cluster import KMeans
12 from sklearn.metrics import (
13     silhouette_score, accuracy_score, precision_score,
14     recall_score, f1_score, roc_auc_score, roc_curve,
15 )
16 from sklearn.linear_model import LogisticRegression
17 from sklearn.ensemble import RandomForestClassifier,
18     GradientBoostingClassifier
19 from sklearn.model_selection import train_test_split, StratifiedKFold,
20     cross_val_score
21 from sklearn.impute import SimpleImputer
22
23 sns.set_theme(style="whitegrid", palette="muted")
24 plt.rcParams.update({"figure.dpi": 120, "axes.titlesize": 13,
25                     "axes.labelsize": 11, "legend.fontsize": 10})
26 SEED = 42
27 np.random.seed(SEED)

```

### A.2 — Step 1: Data Loading & Preprocessing

```

1 DATA_PATH = "../dataverse_files/...CITIZEN_SURVEY_ALL_DATA.dta"
2 df_raw = pd.read_stata(DATA_PATH)
3
4 def _clean_labels(series):
5     return series.astype(str).str.strip().str.lower()
6
7 def map_binary_labels(series, positive_labels, negative_labels):

```

```

8     """Map string-labeled categories to {1, 0, NaN}."""
9     cleaned = _clean_labels(series)
10    pos = {x.lower() for x in positive_labels}
11    neg = {x.lower() for x in negative_labels}
12    out = pd.Series(np.nan, index=series.index, dtype=float)
13    out.loc[cleaned.isin(pos)] = 1.0
14    out.loc[cleaned.isin(neg)] = 0.0
15    return out
16
17    def map_ordinal_labels(series, mapping):
18        """Map string-labeled categories to ordered numeric scores."""
19        cleaned = _clean_labels(series)
20        mapping = {k.lower(): float(v) for k, v in mapping.items()}
21        return cleaned.map(mapping).astype(float)
22
23    # Encode TARGET: c17_f
24    df["telemedicine_usage"] = map_binary_labels(
25        df["c17_f"],
26        positive_labels={"Great Extent", "Somewhat"},
27        negative_labels={"Very Little"},
28    )
29
30    # Encode binary predictors
31    df["c16_g"] = map_binary_labels(df["c16_g"],
32        positive_labels={"Yes (Spontaneous)", "Yes (Prompted)"},
33        negative_labels={"No"})
34
35    for col in ["c17_a", "c17_b", "c17_c", "c17_d", "c17_e"]:
36        df[col] = map_binary_labels(df[col],
37            positive_labels={"Great Extent", "Somewhat"},
38            negative_labels={"Very Little"})
39
40    df["gender"] = map_binary_labels(df["a2c"],
41        positive_labels={"Female"}, negative_labels={"Male"})
42    df["rural"] = map_binary_labels(df["residence"],
43        positive_labels={"Rural"}, negative_labels={"Urban"})
44    df["c18_a"] = map_binary_labels(df["c18_a"],
45        positive_labels={"Yes"}, negative_labels={"No"})
46
47    EDU_ORDER = {"primary": 1, "middle": 2, "secondary": 3, "higher-
48        secondary+": 4}
49    df["education"] = map_ordinal_labels(df["rec_a2d"], EDU_ORDER)
50
51    # Subset, impute, standardise
52    df_work = df[PREDICTOR_COLS + ["telemedicine_usage"]].dropna(
53        subset=["telemedicine_usage"])
54
55    df_work[continuous_vars] = SimpleImputer(strategy="median").
56        fit_transform(
57            df_work[continuous_vars])
58
59    df_work[ordinal_binary_vars] = SimpleImputer(strategy="most_frequent").
60        fit_transform(
61            df_work[ordinal_binary_vars])
62
63    scaler = StandardScaler()
64    X_scaled = scaler.fit_transform(df_work[PREDICTOR_COLS])

```

### A.3 — Step 2: Digital Readiness Score (DRS)

```

1 DRS_COLS = ["c16_g", "c17_a", "c17_b", "c17_c", "c17_d", "c17_e"]
2 X_drs = df_scaled[DRS_COLS].values
3
4 pca_drs = PCA(n_components=len(DRS_COLS), random_state=SEED)
5 pca_drs.fit(X_drs)
6
7 ev = pca_drs.explained_variance_ratio_
8 pc1_variance = ev[0]
9
10 if pc1_variance >= 0.60:
11     drs_values = pca_drs.transform(X_drs)[: , 0]
12 else:
13     # Variance-weighted composite
14     scores = pca_drs.transform(X_drs)
15     drs_values = (scores * ev).sum(axis=1)
16
17 drs_values = (drs_values - drs_values.mean()) / drs_values.std()
18 df["DRS"] = drs_values
19 df_scaled["DRS"] = drs_values

```

### A.4 — Step 3: Full-Predictor PCA

```

1 pca_full = PCA(n_components=None, random_state=SEED)
2 pca_full.fit(df_scaled[PREDICTOR_COLS].values)
3
4 cumvar = np.cumsum(pca_full.explained_variance_ratio_)
5 n_components_80 = int(np.searchsorted(cumvar, 0.80)) + 1
6
7 pca_final = PCA(n_components=n_components_80, random_state=SEED)
8 Z = pca_final.fit_transform(df_scaled[PREDICTOR_COLS].values)

```

### A.5 — Step 4: K-Means Clustering

```

1 K_RANGE = range(2, 8)
2 wcss, sil_scores = [], []
3 sil_idx = np.random.choice(N, min(5000, N), replace=False)
4 Z_sample = Z[sil_idx]
5
6 for k in K_RANGE:
7     km = KMeans(n_clusters=k, init="k-means++", n_init=10,
8                 max_iter=300, random_state=SEED)
9     km.fit(Z)
10    wcss.append(km.inertia_)
11    labels_sample = km.labels_[sil_idx]
12    sil = silhouette_score(Z_sample, labels_sample, random_state=SEED)
13    sil_scores.append(sil)
14
15 K_OPT = K_RANGE.start + int(np.argmax(sil_scores))

```

```

16 km_final = KMeans(n_clusters=K_OPT, init="k-means++", n_init=20,
17                   max_iter=500, random_state=SEED)
18 cluster_labels = km_final.fit_predict(Z)
19 df["cluster"] = cluster_labels

```

## A.6 — Steps 5–7: Classifiers

```

1 # Interaction terms
2 df_model["gender_x_rural"] = df["gender"] * df["rural"]
3 df_model["income_x_internet"] = df_scaled["a2g_food"] * df["c16_g"]
4 cluster_dummies = pd.get_dummies(df["cluster"], prefix="cluster",
5                                  drop_first=True)
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.20, stratify=y, random_state=SEED)
9
10 # Logistic Regression
11 lr = LogisticRegression(C=1.0, solver="lbfgs", max_iter=500,
12                        class_weight="balanced", random_state=SEED)
13 lr.fit(X_train, y_train)
14
15 # Random Forest
16 rf = RandomForestClassifier(
17     n_estimators=200, max_depth=8, max_features="sqrt",
18     class_weight="balanced", oob_score=True,
19     n_jobs=-1, random_state=SEED)
20 rf.fit(X_train, y_train)
21
22 # Gradient Boosting
23 gb = GradientBoostingClassifier(
24     n_estimators=200, learning_rate=0.05,
25     max_depth=4, subsample=0.8, random_state=SEED)
26 gb.fit(X_train, y_train)

```

## A.7 — Step 8: Model Validation & Comparison

```

1 def compute_metrics(y_true, y_pred, y_prob, name):
2     return {
3         "Model" : name,
4         "Accuracy" : round(accuracy_score(y_true, y_pred), 4),
5         "Precision" : round(precision_score(y_true, y_pred, zero_division
6         =0), 4),
7         "Recall" : round(recall_score(y_true, y_pred, zero_division=0)
8         , 4),
9         "F1" : round(f1_score(y_true, y_pred, zero_division=0), 4)
10        ,
11        "ROC-AUC" : round(roc_auc_score(y_true, y_prob), 4),
12    }
13
14 def bootstrap_auc_ci(y_true, y_prob, B=1000, seed=SEED):
15     rng = np.random.RandomState(seed)
16     aucs = []
17     for _ in range(B):

```



```

15     idx = rng.choice(len(y_true), size=len(y_true), replace=True)
16     if len(np.unique(y_true[idx])) < 2:
17         continue
18     aucs.append(roc_auc_score(y_true[idx], y_prob[idx]))
19     return np.percentile(aucs, [2.5, 97.5])
20
21 # 5-fold CV
22 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
23 for model, name in [(lr, "LR"), (rf, "RF"), (gb, "GB")]:
24     scores = cross_val_score(model, X_train, y_train,
25                             cv=cv, scoring="roc_auc", n_jobs=-1)
26     print(f"{name}: {scores.mean():.4f} +/- {scores.std():.4f}")
27
28 # Bootstrap CIs
29 for y_prob, name in [(y_prob_lr, "LR"), (y_prob_rf, "RF"),
30                     (y_prob_gb, "GB")]:
31     lo, hi = bootstrap_auc_ci(y_test, y_prob)
32     auc = roc_auc_score(y_test, y_prob)
33     print(f"{name}: AUC={auc:.4f} [95% CI: {lo:.4f}-{hi:.4f}]")

```

## B. History of AI Prompts and Chats

The following table summarises the AI-assisted interactions undertaken during the project. All AI-generated content was critically reviewed, modified, and integrated with independent judgment by the team.

**Table 10.** Summary of AI prompt interactions

| # | Prompt / Query                                                                                     | AI Response Summary                                                                                                                                                           | Usage in Project                                                                                 |
|---|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 1 | “How do I robustly read Stata-labelled categorical columns in pandas without losing value labels?” | Suggested converting to string, stripping whitespace, using <code>.lower()</code> , and implementing custom map functions to handle the mixed label formats from Stata files. | Implemented in <code>map_binary_labels()</code> and <code>map_ordinal_labels()</code> in Step 1. |
| 2 | “Why is my PCA-based DRS giving NaN values?”                                                       | Identified that degenerate (constant) columns after preprocessing could cause PCA to fail. Suggested adding a guard check for zero-variance columns before fitting.           | Added <code>np.std</code> guard check in Step 2; see DRS code block.                             |

| # | Prompt / Query                                                                              | AI Response Summary                                                                                                                                                                                                                               | Usage in Project                                                                                                    |
|---|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 3 | “What is the best practice for K-Means on large datasets for silhouette computation?”       | Recommended subsampling (5,000 obs) for silhouette score computation to reduce quadratic time complexity from $O(n^2)$ to manageable runtime without significant accuracy loss.                                                                   | Implemented in Step 4; silhouette computed on random subsample of up to 5,000 observations.                         |
| 4 | “How should I handle class imbalance (35%/65%) in a logistic regression and RF classifier?” | Advised using <code>class_weight='balanced'</code> in scikit-learn, which adjusts weights inversely proportional to class frequencies, and using ROC-AUC rather than accuracy as the primary metric.                                              | Applied to all three classifiers; ROC-AUC used as primary metric in Step 8.                                         |
| 5 | “How do I compute a proper bootstrap confidence interval for ROC-AUC?”                      | Provided the percentile bootstrap method: resample $(y, \hat{p})$ pairs $B = 1000$ times with replacement; compute AUC on each resample; take 2.5th and 97.5th percentiles. Warned about handling edge cases where resampled class is degenerate. | Implemented in <code>bootstrap_auc_ci()</code> in Step 8.                                                           |
| 6 | “How do I interpret the ‘DK’ and ‘DNA’ response categories in the target variable?”         | Explained that DK (Don’t Know) and DNA (Does Not Apply) represent forms of non-response that should not be coded as 0 or 1; recommended treating them as missing (NaN) and excluding from the target.                                             | Applied in target encoding: only “Great Extent”, “Somewhat”, and “Very Little” are retained as valid target values. |
| 7 | “What is the mathematical relationship between PCA eigenvalues and explained variance?”     | Derived that the proportion of variance explained by $PC_k$ is $\lambda_k / \sum_j \lambda_j$ , where $\lambda_k$ are the eigenvalues of the covariance matrix.                                                                                   | Used to verify DRS variance calculations in Step 2 and document Section 3.2.                                        |

| #  | Prompt / Query                                                                      | AI Response Summary                                                                                                                                                                        | Usage in Project                                                      |
|----|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 8  | “How do I create a variance-weighted composite score from multiple PCA components?” | Suggested computing weighted sum of all component scores using explained variance ratios as weights: $\sum_k (\lambda_k / \sum_j \lambda_j) \cdot \mathbf{v}_k^\top \mathbf{x}$ .          | Implemented as fallback DRS when PC1 explains less than 60% variance. |
| 9  | “Review my logistic regression odds ratio interpretation”                           | Confirmed that <code>np.exp(lr.coef_[0])</code> gives odds ratios; noted that $OR > 1$ means increased odds of $Y = 1$ and $OR < 1$ means decreased odds, controlling for other variables. | Interpretive framework in Section 5.3 and Table 5.                    |
| 10 | “How do I produce the elbow plot and silhouette plot for K-Means in Python?”        | Provided matplotlib code template with dual subplots: WCSS vs $K$ on left panel, silhouette score vs $K$ on right panel, with a vertical line at the optimal $K$ .                         | Adapted for Figure 2 in Step 9 (Visualisations).                      |

### C. AI-Code Correspondence: One-to-One Mapping

This appendix provides an explicit one-to-one correspondence between AI-generated outputs and where they are used in the final report and code. This satisfies the transparency requirement for AI-assisted academic work.

**Table 11.** One-to-one correspondence: AI output  $\rightarrow$  code/report location

| # | AI-Generated Output                                                                                                 | Where Used in Code                                                                                                     | Where Used in Report                                                  |
|---|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 1 | Pattern for robust Stata categorical label handling using <code>str.strip().lower()</code> and custom map functions | Functions <code>map_binary_labels()</code> and <code>map_ordinal_labels()</code> in Step 1 (lines 146–161 of notebook) | Preprocessing description in Section 3.1; Step 1 code in Appendix A.2 |
| 2 | Suggestion to add zero-variance guard before PCA                                                                    | <code>if np.all(np.std(X_drs, axis=0) &lt; 1e-12): raise ValueError(...)</code> in Step 2                              | Section 3.2 assumption note; DRS code block in Appendix A.3           |

| #  | AI-Generated Output                                                                                                           | Where Used in Code                                                                                                                                              | Where Used in Report                                                            |
|----|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 3  | Recommendation to subsample for silhouette score computation to manage $O(n^2)$ complexity                                    | <code>sil_idx = np.random.choice(N, min(5000, N), replace=False)</code> in Step 4                                                                               | K-Means implementation details in Section 3.4; clustering code in Appendix A.5  |
| 4  | <code>class_weight='balanced')</code> advice for imbalanced classes; ROC-AUC as primary metric                                | Applied to <code>LogisticRegression</code> , <code>RandomForestClassifier</code> constructors; ROC-AUC used as primary metric in <code>compute_metrics()</code> | Section 3.7 (model validation); Table 7 primary metric note                     |
| 5  | Percentile bootstrap CI implementation for ROC-AUC, including edge case handling                                              | <code>bootstrap_auc_ci()</code> function in Step 8 (lines 811–820 of notebook)                                                                                  | Section 3.7; Table 9; Appendix A.7                                              |
| 6  | Classification of DK/DNA responses as missing (not 0 or 1) for target encoding                                                | Target encoding logic in Step 1: only “Great Extent”, “Somewhat”, “Very Little” mapped to $\{1, 0\}$                                                            | Section 4.2 (target variable description); Section 3.1                          |
| 7  | Mathematical derivation of PCA explained variance ( $\lambda_k / \sum \lambda_j$ )                                            | Used to construct <code>ev = pca_drs.explained_variance_ratio_</code> and variance-weighted composite in Step 2                                                 | Equations (1)–(3) in Section 3.2; interpretation of PC1 variance in Section 5.1 |
| 8  | Variance-weighted composite formula: $\sum_k w_k \cdot \mathbf{v}_k^\top \mathbf{x}$ where $w_k = \lambda_k / \sum \lambda_j$ | <code>drs_values = (scores * ev).sum(axis=1)</code> in Step 2 when PC1 < 60%                                                                                    | Equation (3) in Section 3.2; DRS results in Section 5.1                         |
| 9  | Confirmation of OR interpretation: $e^{\hat{\beta}_j}$ gives multiplicative change in odds                                    | <code>odds_ratios = np.exp(lr.coef_[0])</code> in Step 5; sort by <code>abs(OR-1)</code>                                                                        | Section 3.5 interpretation paragraph; Table 5 in Section 5.3                    |
| 10 | Dual-subplot matplotlib template for elbow and silhouette plots                                                               | Figure 2 code in Step 9 ( <code>fig2, axes = plt.subplots(1, 2, ...)</code> )                                                                                   | Referenced as Figure 2 in Section 5.2 (would appear in Appendix D)              |

**Transparency note:** In all cases above, AI outputs were used as *starting points or sanity checks*. All final modeling decisions (choice of  $K$ , threshold for DRS fallback, interaction terms, hyperparameter values) were made independently by the team. No AI output was used verbatim in the report text without modification and critical review.

## D. Additional Technical Details and Supplementary Information

### D.1 — Dataset Summary Statistics

**Table 12.** Analytic sample summary: key variable statistics ( $n = 30,629$ )

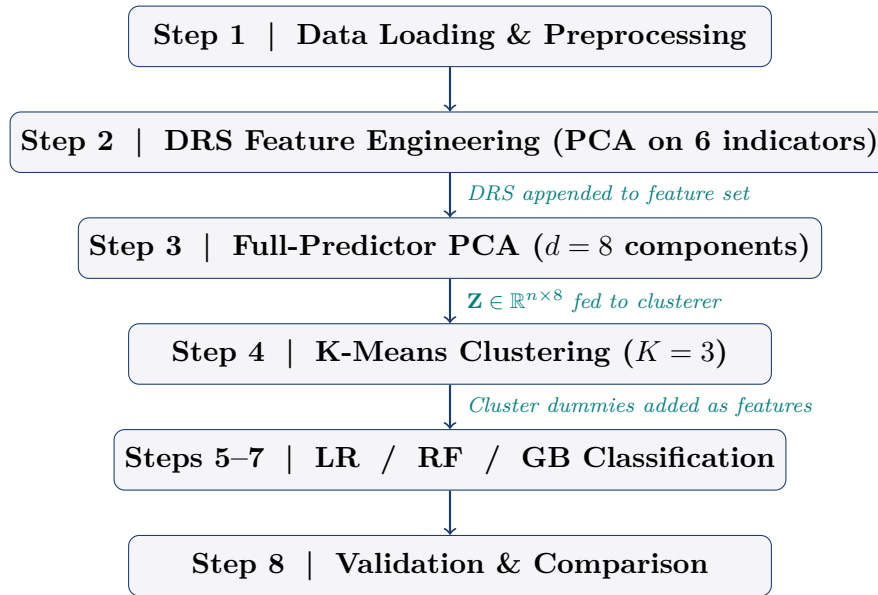
| Variable                          | Mean  | Std Dev | Min   | Max  |
|-----------------------------------|-------|---------|-------|------|
| Telemedicine usage ( $Y$ )        | 0.346 | 0.476   | 0     | 1    |
| Gender (Female=1)                 | 0.437 | 0.496   | 0     | 1    |
| Rural (=1)                        | 0.706 | 0.456   | 0     | 1    |
| Education (1–4 ordinal)           | 2.38  | 1.12    | 1     | 4    |
| Internet as health source (c16_g) | 0.639 | 0.480   | 0     | 1    |
| c17_e (treatment help)            | 0.765 | 0.424   | 0     | 1    |
| Digital Readiness Score (DRS)     | 0.000 | 1.001   | −2.85 | 1.95 |

### D.2 — Full Cluster Profile Table

**Table 13.** Cluster profiles: mean values of original predictor variables

| Variable                | Cluster 0 | Cluster 1    | Cluster 2 |
|-------------------------|-----------|--------------|-----------|
| Telemedicine usage rate | 0.377     | <b>0.469</b> | 0.059     |
| c17_e (treatment help)  | 0.839     | <b>0.899</b> | 0.512     |
| c16_g (internet source) | 0.812     | <b>0.865</b> | 0.242     |
| Education               | 2.51      | 2.42         | 2.27      |
| Rural                   | 0.721     | 0.694        | 0.730     |
| % of sample             | 7.2%      | 64.3%        | 28.5%     |

### D.3 — Pipeline Architecture



### D.4 — Figure Export Reference

All figures are generated in Step 9 of `CADS_Project.ipynb` and appear consolidated in the **Figures** section preceding the Conclusions. To export from the notebook, add the following `savefig` calls immediately before each `plt.show()` in Step 9:

```

1 fig1.savefig("fig1_drs_scree.pdf",          bbox_inches="tight", dpi=150)
2 fig2.savefig("fig2_kmeans_selection.pdf",   bbox_inches="tight", dpi=150)
3 fig3.savefig("fig3_pca_biplot.pdf",         bbox_inches="tight", dpi=150)
4 fig4.savefig("fig4_cluster_heatmap.pdf",    bbox_inches="tight", dpi=150)
5 fig5.savefig("fig5_odds_ratios.pdf",        bbox_inches="tight", dpi=150)
6 fig6.savefig("fig6_rf_importance.pdf",      bbox_inches="tight", dpi=150)
7 fig7.savefig("fig7_roc_curves.pdf",         bbox_inches="tight", dpi=150)
8 fig8.savefig("fig8_model_comparison.pdf",   bbox_inches="tight", dpi=150)

```

### D.5 — Computational Environment

| Component    | Version / Detail                            |
|--------------|---------------------------------------------|
| Python       | 3.12.4                                      |
| scikit-learn | used for PCA, K-Means, classifiers, CV      |
| pandas       | data manipulation and encoding              |
| numpy        | numerical operations, random seed (SEED=42) |
| statsmodels  | GLM estimation reference                    |
| matplotlib   | visualisation (dpi=120)                     |
| seaborn      | statistical graphics (whitegrid theme)      |